EuroSAT Data Mining and Analysis Project

Nitai Shah Jvk Chaitanya Shreya Muppidi Jeffrey Joseph Ayaan Omair

***Abstract*—This report explores the EuroSAT dataset, which comprises 27,000 multispectral satellite images spanning 10 distinct land use and land cover classes, including Forest, Annual Crop, and Residential. Captured by Sentinel-2, the dataset leverages 13 spectral bands ranging from visible light (RGB) to near-infrared and shortwave infrared, offering rich spatial and spectral details. We conducted experiments comparing machine learning and deep learning approaches for classification. A baseline Random Forest model achieved an accuracy of 79.69%, while a convolutional neural network (CNN) outperformed it with a test accuracy of 86.46%, effectively leveraging spatial and spectral patterns. Further analysis demonstrated the added predictive value of multispectral data over traditional RGB imagery, confirming the significance of using all available bands for enhanced accuracy. These results highlight the utility of the EuroSAT dataset and the potential of deep learning for advancing land-use classification tasks.**

I. Introduction

The EuroSAT dataset includes imagery categorized into 10 distinct land cover classes: Industrial, Residential, Annual Crop, Permanent Crop, River, Sea & Lake, Herbaceous Vegetation, Highway, Pasture, and Forest. Each class contains a varying number of images corresponding to its specific category, collectively amounting to a total of 27,000 images. These images are derived from Sentinel-2 satellite data and span 13 spectral bands, enabling a comprehensive analysis of land cover types. The table below highlights the distinctions between the different spectral bands:

Band	Resolution (m)	Central Wavelength (nm)	Description
B1	60	443	Coastal aerosol
B2	10	490	Blue
B3	10	560	Green
B4	10	665	Red
B5	20	705	Red Edge 1
B6	20	740	Red Edge 2
B7	20	783	Red Edge 3
B8	10	842	Near Infrared (NIR)
B8A	20	865	Narrow Near Infrared
B9	60	945	Water vapor
B10	60	1375	Cirrus
B11	20	1610	Shortwave Infrared 1 (SWIR1)
B12	20	2190	Shortwave Infrared 2 (SWIR2)

TABLE I: Spectral Band table

In this study, we conducted multiple experiments to evaluate the effectiveness of the learning models for land cover classification using the EuroSAT dataset. The initial experiment involved establishing a baseline accuracy using a Random Forest classifier, which was followed by training a Convolutional Neural Network (CNN) on the full 13-band multispectral data. We also performed a comparative experiment by training the same CNN model using only RGB data to assess the additional value provided by multispectral bands.

Literature Review:

The EuroSAT dataset has been widely used in the field of remote sensing for land use and land cover classification. Helber et al. [1] introduced the dataset, which includes satellite images with various spectral bands. Research has shown that Convolutional Neural Networks (CNNs) are particularly well-suited for classifying satellite imagery due to their ability to capture spatial patterns. For example, Chen et al. demonstrated the effectiveness of CNNs in multispectral land cover classification, while several other studies have highlighted the importance of spectral bands such as Near-Infrared (NIR) and Shortwave Infrared (SWIR) for distinguishing different land cover types.

This study highlights the effectiveness of CNNs in classifying land cover types using satellite imagery. The findings underscore the value of multispectral data in improving classification accuracy, with implications for environmental monitoring, urban planning, and disaster management.

Exploratory Data Analysis

When performing EDA, we first looked at the distribution for each class in the dataframe. The following image portrays the class distribution:

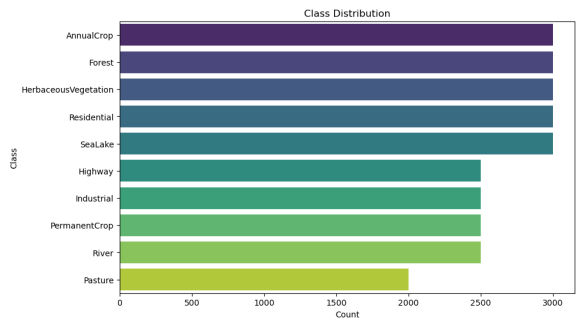


Fig. 1: Distribution for Each Class

This plot illustrates the distribution of images across different classes. The Annual Crop, Forest, Residential, and Herbaceous Vegetation classes have the highest number of images, each with approximately 3,000.
To analyze the class distribution further, we calculated the average pixel intensity for each image. This was achieved by converting each image into an array using the *imread()* function. Since the images are 64x64, this process produced 64 arrays, each containing 13 values corresponding to the 13 spectral bands of the images. We then computed the mean of all these arrays for each image. Finally, the average pixel intensities were plotted for each image class. The following histograms illustrate the pixel intensity distribution for a few of the classes.

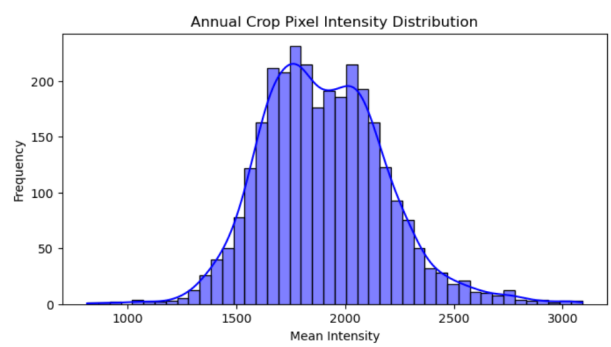


Fig. 2: Annual Crop Pixel Intensity Distribution

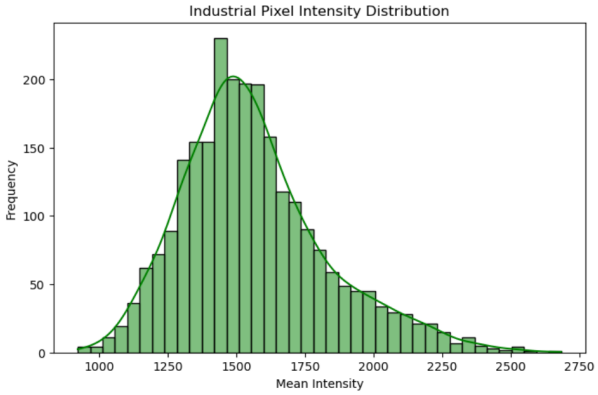


Fig. 3: Industrial Pixel Intensity Distribution

II. Method

The Eurosat Dataset has each individual class arranged in separate folders, with each folder containing the corresponding class images, hence the first step of our project was designing an I/O pipeline. The pipeline starts with reading each image path from a particular file, and assigning a class label such as 0 for Annual Crop, 1 for Forest and so on for all 13 classes. Next we define a function that takes our file path and reads in the image as an array using the RasterIO library. The output array is of shape (13, 64, 64), where 13 represents the number of bands present in the image and 64,64 represents the height and width of the image. Then we normalise our images by dividing each pixel by the maximum pixel value, to speed up convergence during training. For this project we have chosen to work with Convolutional Neural Networks (CNN's). CNN models are particularly suited for image data since they can successfully capture the spatial and hierarchical patterns present in an image. Additionally a CNN model could fully leverage the different spectral bands present in the dataset, allowing us to differentiate between different materials. The following is a description of the CNN model architecture.

Layer	Output Shape	Description
Input Layer	(64, 64, 13)	Input image with dimensions 64x64 and 13 spectral bands.
Conv2D (Block 1)	(62, 62, 32)	32 filters of size 3x3, ReLU activation.
MaxPooling2D	(31, 31, 32)	Downsamples the spatial dimensions by a factor of 2.
Conv2D (Block 2)	(29, 29, 64)	64 filters of size 3x3, ReLU activation.
MaxPooling2D	(14, 14, 64)	Downsamples the spatial dimensions by a factor of 2.
Conv2D (Block 3)	(12, 12, 128)	128 filters of size 3x3, ReLU activation.
Flatten	(18,432)	Converts the 3D feature maps into a 1D vector.
Dense (Hidden Layer)	(64)	Fully connected layer with 64 neurons, ReLU activation.
Dense (Output Layer)	(13)	Fully connected output layer with 13 neurons, softmax activation.

TABLE II: Layer Configuration

Now we shall evaluate the training strategy utilised for the project. Firstly we divide our datasets into training (60%), testing(20%) and validation (20%) splits. Next we define the architecture as in the above table, choosing an activation function of ReLU for hidden units, and softmax for the output layer, since we are performing multi-class classification. We then select the Adam optimizer as our optimizer of choice, since it is computationally efficient and works well across multiple domains. For our loss function we chose the sparse categorical cross-entropy loss, since it doesn't require us to perform One-Hot-Encoding, and can directly use the class labels (1,2,3, etc). Then we set an epoch size of 10, striking a balance between computation time and accuracy. For similar reasons, we also choose a batch size of 32. The model is then compiled and trained on the training set, and the corresponding metrics are displayed for the training and validation sets. We shall use 4 metrics to evaluate our classifier. These are Accuracy, Precision, Recall and F1-Score.
Accuracy measures the proportion of correctly predicted samples out of all samples.
Accuracy = Number of correct predictions / Total Number of Predictions
Precision measures the proportion of true positive predictions out of all positive predictions made by the model.
Precision = TP/(TP + FP)
Recall measures the proportion of actual positives that were correctly identified by the model.
Recall = TP / (TP + FN)
F1 score is the harmonic mean of precision and recall. It provides a single metric that balances both precision and recall.
F1 score = Precision*Recall / (Precision + Recall)

III. Experimental Results and Discussion

A. *Experiment 1 : Exploring Different Model Types (CNN and Random Forest)*

The first experiment that we carried out as part of our project was the application of baseline Machine Learning and Deep Learning Models to the Multispectral Dataset. To get a baseline accuracy on our dataset we chose the Random Forest Model because of its versatility along with its ease of implementation. Since our data preparation pipeline is suited for 2D array input, we flattened the 2D array into a 1D vector and then passed it to the Random Forest Model, using the default hyper-parameter values provided by Scikit-Learn. We obtained a Test Set Accuracy of 0.7969, a Precision of 0.7969, a Recall of 0.7969 and a F1-Score of 0.7969 as well. The similarity of all these metrics could be attributed to an almost perfect class balance in our dataset. Now that we had a baseline model ready, we began training our initial CNN model, with the architecture and strategy described in the methods section. The following table displays the Training, Validation and Testing Metrics on the data . The Training/Validation accuracy and loss curves are shown as well along with the confusion matrix on the Test Set. The following table displays the Training, Validation and Testing Metrics on the data . The Training/Validation accuracy and loss curves are shown as well along with the confusion matrix on the Test Set

Set	Accuracy	Precision	Recall	F1-Score
Train	0.9329	0.9329	0.9329	0.9329
Val	0.8670	0.8670	0.8670	0.8670
Test	0.8646	0.8646	0.8646	0.8646

TABLE III: Experiment 1 Evaluation Metrics for Train, Validation, and Test Sets

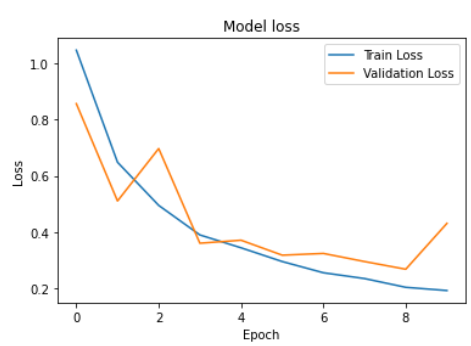


Fig. 4: Experiment 1 Model Loss plot

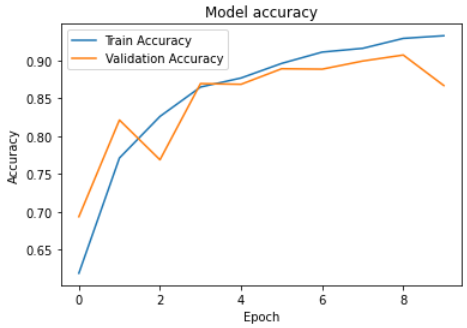


Fig. 5: experiment 1 Model Accuracy Plot

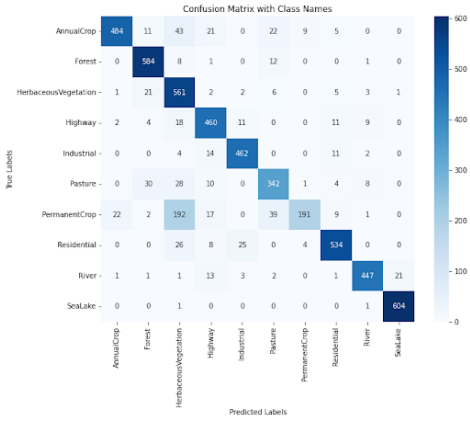


Fig. 6: Experiment 1 Confusion Matrix

From the evaluation metrics we see that our CNN Model obtains a 86.46% accuracy on the Test Data. This is significantly better than the baseline Random Forest Model. Thus we can conclude that the CNN model is able to leverage spatial patterns in the data and obtain a better accuracy. This also leads us to the next experiment, on whether the multispectral data itself is contributing to the increase in accuracy. These models could be used for tasks such as Land Cover Estimation. For example, this model could be used to classify what percentage of a city has been taken over by residential/industrial use.

B. *Experiment 2 : Training on only RGB instead of multispectral*

From the previous experiments we observed that the CNN model beat the RF model in accuracy. One of the possible explanations was the CNN model architecture. The second possibility arises from the data itself, which is multispectral in nature. In a normal RGB image, we only have 3 bands, whereas in a multispectral image we have multiple other bands such as Near Infrared (NIR), Short Wave Infrared, etc. These bands add more information to the model, allowing us to better differentiate between classes. For example the NIR band reflects more vegetation, as compared to other classes such as Industrial, Sea, etc. This extra "dimension" gives the deep learning model more patterns to learn from. To test our assumption we trained the same CNN model in experiment 1, however only with RGB data. The results are displayed below

Set	Accuracy	Precision	Recall	F1-Score
Train	0.8527	0.8527	0.8527	0.8527
Val	0.8069	0.8069	0.8069	0.8069
Test	0.8141	0.8141	0.8141	0.8141

TABLE IV: Experiment 2: Evaluation Metrics for Train, Validation, and Test Sets

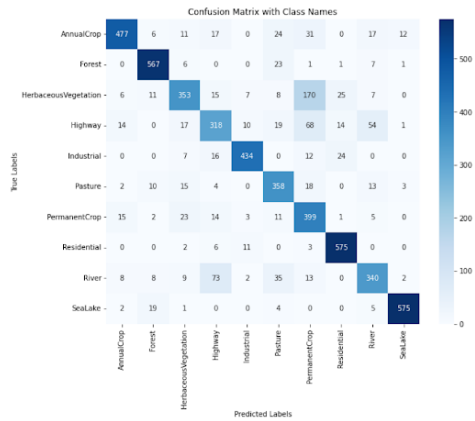


Fig. 7: Experiment 2 Confusion Matrix

From the error metrics, we do see that the RGB only trained model performs worse than the Multi-Spectral Model, validating our assumption. In addition, we see that the RGB-CNN model still outperforms the baseline RF model, validating the assumption that a CNN architecture is well suited for Image Data, as compared to traditional ML models.

C. *Experiment 3: Zero Shot Classification*

For this experiment, we made use of Zero Shot Classification to classify the different classes present in the dataset. Zero Shot Classification is an approach which aims to classify into categories it has never seen during training. They work by leveraging general knowledge to associate input images with the associated class labels. Hence the output of our model, depends on the number of classes we pass to it. We make use of OpenAi's CLIP-vit-base-patch32 model, loaded via Py-Torch. Then we define the associated class labels for the dataset, and pass 20% of the data from each class to the CLIP model. The metrics and confusion matrix are as follows.

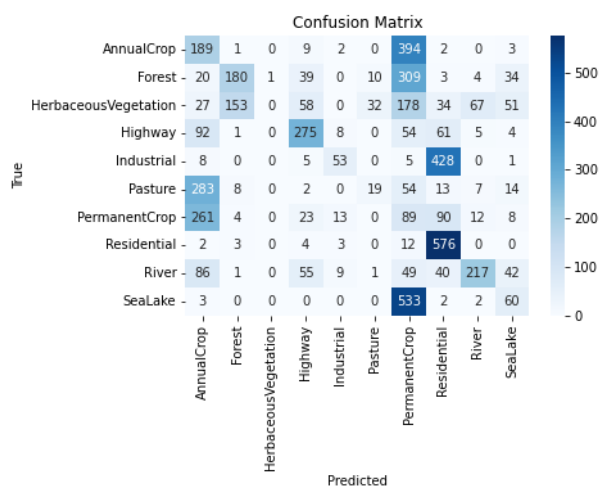


Fig. 8: Zero Shot Classification Confusion Matrix

Metric	Value
Accuracy	0.3070
Precision	0.3621
Recall	0.3070
F1-Score	0.2866

TABLE V: Performance Metrics

As seen from the above table and confusion matrix the classification accuracy for the model isn't as good as the previous CNN models. This could potentially be due to the domain gap, since CLIP is trained on diverse internet data that may not align with the Eurosat Dataset. In addition to this the Eurosat dataset has some images that include multiple classes in a single image, causing further errors. To mitigate this issue we decided to clip similar classes into 1 group. For example, the classes Annual Crop, Forest, Herbaceous Vegetation, Pasture and Permanent Crop were grouped into 1 category - Vegetation. Similarly, Highways, Industrial and Residential were grouped into Built areas and Rivers, Seas and Lakes were grouped into Water Bodies. The model's performance was then evaluated using these classes and the metrics displayed as below.

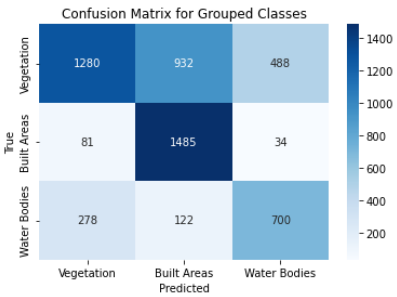


Fig. 9: Grouped Classes Confusion Matrix

Metric	Value
Accuracy	0.6417
Precision	0.6805
Recall	0.6417
F1-Score	0.6304

TABLE VI: Grouped classes Performance Metrics

We obtain better accuracy using the grouping strategy, since we reduced class ambiguity and the simplified the classification space.

D. Experiment 4: Conv3D

In this experiment, we evaluated the performance of a classification model on 13-band images using Conv3D layers. Conv3D (3D Convolution) is a type of convolution operation that processes three-dimensional data, making it particularly useful for datasets with spatial-temporal or spatial-depth structures. Unlike Conv2D, which operates on two-dimensional data such as static images, Conv3D uses a three-dimensional kernel that slides across height, width, and depth (or time) to capture spatial and depth-related relationships. This capability makes Conv3D particularly effective for analysing video sequences, volumetric medical images, or multi-band imagery. However, the inclusion of an additional dimension also increases computational complexity compared to Conv2D.

The Conv3D model was designed as a sequential architecture with the following layers:

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv3d (Conv3D)             (None, 128, 128, 13, 32)  896

 max_pooling3d (MaxPooling3D  (None, 64, 64, 6, 32)    0
 )

 conv3d_1 (Conv3D)           (None, 64, 64, 6, 64)     55360

 max_pooling3d_1 (MaxPooling  (None, 32, 32, 3, 64)    0
 3D)

 conv3d_2 (Conv3D)           (None, 32, 32, 3, 128)    221312

 dropout (Dropout)           (None, 32, 32, 3, 128)    0

 conv3d_3 (Conv3D)           (None, 32, 32, 3, 256)    884992

 global_average_pooling3d (G  (None, 256)              0
 lobalAveragePooling3D)

 dense (Dense)               (None, 256)               65792

 dense_1 (Dense)             (None, 128)               32896

 dropout_1 (Dropout)         (None, 128)               0

 dense_2 (Dense)             (None, 64)                8256

 dense_3 (Dense)             (None, 10)                650

=================================================================
Total params: 1,270,154
Trainable params: 1,270,154
Non-trainable params: 0
_________________________________________________________________
```

Fig. 10: Conv3D Model Layers

This architecture was designed to progressively reduce the spatial and depth dimensions while increasing the number of filters to extract high-level features.

To enhance the model's robustness and prevent overfitting, we applied 3D data augmentation techniques, including spatial flipping and rotations along multiple axes. These augmentations increased the diversity of the training dataset without collecting additional data.

We implemented the following callbacks to improve training efficiency:

1) **Learning Rate Scheduler: Applied a linear decay of 5% to the learning rate after each epoch to ensure smooth convergence.**
2) **Model Checkpoint: Saved the model whenever the validation loss improved, with the best model stored at the specified file path.**
3) **Early Stopping: Halted training after 5 epochs with no improvement in validation loss, restoring the weights from the best epoch.**

The model was trained on 30 epochs, with callback functions in place to terminate the execution of epochs if there is no improvement in the model.

Below are the results showing our model performance:

1) Confusion matrix

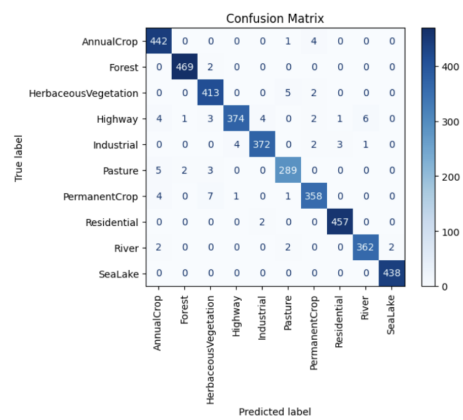
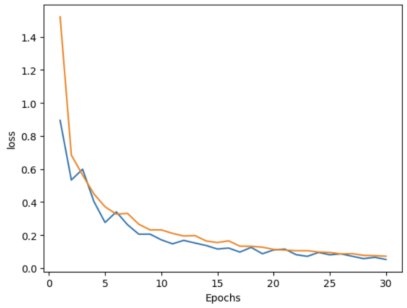


Fig. 11: Confusion Matrix

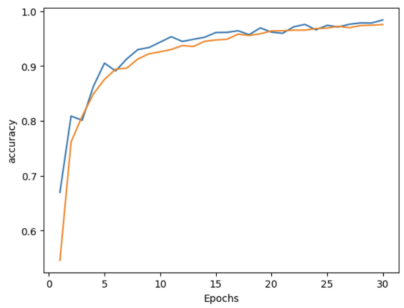
2) Overall classification report:

Metric	Value
Accuracy	98.44%
Precision	98.38%
Recall	98.27%
F1 Score	98.32%

3) Model loss over each epoch:



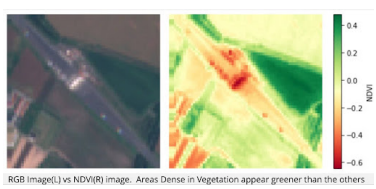
4) Model accuracy over each epoch:



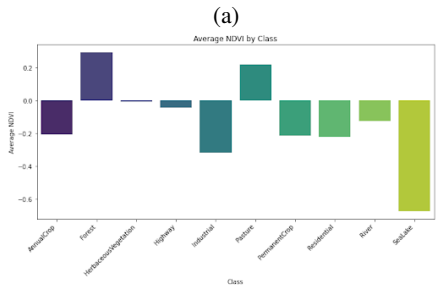
The Conv3D model for classifying 13-band hyperspectral images has proven highly effective, achieving a validation accuracy of approximately **98%**, surpassing Conv2D and other CNN and machine learning models explored in this paper so far. Its ability to capture both spatial and spectral features is key to this success, demonstrating the model's suitability for multi-dimensional data analysis. The 3D convolutional layers learn hierarchical features, from basic spatial and spectral patterns to more complex representations, enabling effective differentiation of land-cover classes such as vegetation, urban areas, and water.

E. Experiment 5: Obtaining NDVI values from the data

Normalized Density Vegetation Index (NDVI) is a remote sensing index used to assess the "quality" of vegetation in a particular area. It is calculated using the Red Band and Near Infrared Band (NIR) Band, with the formula being NDVI = (NIR - RED)/(NIR + RED). Since we have 5 different classes related to vegetation inside our dataset, we used our deep learning model to predict these classes from the test dataset, and then calculate the mean NDVI across distinct classes.



(a)



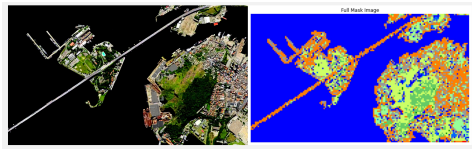
(b)

This can be especially useful to ecologists who can use this index to estimate key issues such as deforestation.

F. Experiment 6: From Classification Output to Segmentation Input on Geo-FS Data

We wanted to test whether our models could perform well on real-world data outside the test set as well. To test this we made use of the Geo-FS flight simulator, modeled on data obtained by the Sentinel Satellite, which is similar to the one used for the EuroSAT dataset. In addition to classification, we wanted to create segmentation masks for the GeoFS data. Since Geo-FS has RGB data, we used the RGB data from the EuroSAT dataset. Firstly we had to make some key modifications to the training data, dividing each 64x64 image into 4 16x16 images. This was done to capture finer details in the image for the segmentation masks. The model was then trained on the RGB Data and saved. Next, we manually captured images from GEO-FS and created an image pre-processing pipeline, that takes in the raw image, calculates its size, and then applies padding so the Height and Width are both divisible by 16. Next, the padded image is then divided into individual tiles of 16x16, after which our RGB model is called to predict the class of each tile. Based on each class we assign a certain color to the entire tile, creating a segmentation mask.

As seen from the image, the model successfully classifies man-made structures represented in orange (bridges, houses, and industries). It can also differentiate between types of vegetation (dense vs light) represented by different shades of green. It also accurately classifies the sea present in the image. Although the output is still rudimentary this experiment demonstrated that our model can generalize to images outside of the dataset as well. This model can be used by city planners to track urban planning and development over the years.



IV. Conclusion

Over the course of this project, we carried out several experiments that took various combinations of our data and tested them with different Machine and Deep Learning Algorithms. In the first experiment, we found that our CNN model beats the Random Forest Model by roughly 8.5 % on the test set. Similarly, in experiment 2 we found that training a model on RGB Data instead of MS Data leads to a decrease in accuracy of 6.2 %. From Experiment 4 we could conclude that using Conv3D leads to a further increase in accuracy over the baseline CNN model by 13.85 %. In experiment 3 we applied **Zero-Shot Classification**, to obtain our class labels, without performing any training. To improve our accuracy, we decided to group similar sub-categories into 1 category. In Experiment 5 we made use of the NDVI index, to display which classes have the highest quality vegetation in them. Finally, in Experiment 6 we made use of data from Geo-FS and coupled it with our RGB model to create image masks that can be used for the task for semantic segmentation. This project allowed us to explore several new concepts such as working with and understanding multi-spectral imagery, convolutional neural networks and zero shot classification. To improve our work we would like to integrate other multi-spectral datasets into our pipeline, to improve our accuracy along with experimenting with different model architectures. We would also like to modify the model we used for Zero-Shot Classification, by re-training the last few layers with the data from the EuroSAT dataset, so it can generalize better. In addition to this we also aim to improve our classification-segmentation pipeline, and build semantic segmentation models to gain more insights into our data.

References

[1] "Papers with Code - EuroSAT: A Novel Dataset and Deep Learning Benchmark for Land Use and Land Cover Classification," Paperswithcode.com, 2017. https://paperswithcode.com/paper/eurosat-a-novel-dataset-and-deep-learning (accessed Nov. 20, 2024).

[2] "Github Link for code: https://github.com/JvkChaitanya/EuroSAT